

Object Oriented Programming

Prof. Mahipal Khoja

Assistant Professor

Artificial Intelligence and Data Science Department

Learning Objectives

1. To introduce the fundamental shift from Procedural Oriented Programming (POP) to **Object-Oriented Programming (OOP)**, highlighting the principles, real-world applications, and historical context of OOP languages.
2. To illustrate the essential **structure of a C++ program**, including the character set, preprocessor directives, and the core "skeleton" required to execute basic logic.
3. To explain the hierarchy of **C++ data types** (Basic, User-defined, and Derived) and the various methods
4. o familiarize students with standard and C++ specific **operators**
5. To demonstrate the importance of data hiding through **Access Specifiers** (Public, Private, Protected),

Learning Outcomes

Students will be able to:

1. **Differentiate** between POP and OOP paradigms by identifying key characteristics
2. **Construct** a well-structured C++ program that utilizes various **Basic, User-defined, and Derived data types**
3. **Apply** the concept of **reference variables** to create aliases for existing data and use **dynamic initialization** to assign values to variables at runtime based on logic or user input.
4. **Resolve** naming conflicts between global and local variables and define class member functions externally
5. **Implement** robust data encapsulation by selecting the appropriate **Access Specifier** (Public, Private, or Protected)

Content

1. Introduction.....	05
2. Difference between OOP and POP.....	12
3. Applications of Object Oriented Programming.....	15
4. Structure of C++ program.....	16
5. Declaration of variables.....	21
6. Basic Data types in C++.....	28
7. Derived Data types.....	32
8. User Defined Data types.....	33
9. Reference variables.....	36
12. Operators in C++.....	38
13. Access Specifiers.....	47

Introduction

- Computer follows an Input-Process-Output cycle. It needs a set of instructions called program to specify the input required, process to be followed, and the output required.
- Program is written in a specific language called programming language, so that computer can understand the instructions.
- Program is a set of instructions to perform a particular job.
- Instructions in a program can be:
 - Sequential
 - Decision-related
 - Iterative

Programming Approaches

➤ Sequential approach:

- It includes programs that are actually a list of instructions to the system used to perform a specific task.
- The limitation with this approach is that with an increase in the number of instructions, the program becomes too lengthy and complicated to understand.

➤ Modular approach:

- It divides the programs into entities known as functions.
- The functions are clubbed together in logical groups with each group containing interrelated functions. Such logical groups are known as modules. It does not portray the real world and entities in the programs

Programming Approaches

- Object-oriented approach:
 - It is the latest of the programming approaches and portrays the real world in the programs.
 - It treats each entity as an object which has a well-defined state, behavior, and a unique identity.
 - It hides unnecessary details from the users and allows creation of programs that are easy to manage and maintain.

Introduction to C++

- The prime purpose of C++ programming was to add object orientation to the C programming language, which is in itself one of the most powerful programming languages.
- The core of the pure object-oriented programming is to create an object, in code, that has certain properties and methods. While designing C++ modules, we try to see whole world in the form of objects
- For example a car is an object which has certain properties such as color, number of doors, and the like. It also has certain methods such as accelerate, brake, and so on.

Keywords/ Characteristics of OOP

Object

- This is the basic unit of object oriented programming. That is both data and function that operate on data are bundled as a unit called as object.

Class

- When you define a class, you define a blueprint for an object. This doesn't actually define any data, but it does define what the class name means, that is, what an object of the class will consist of and what operations can be performed on such an object.

Abstraction

- Data abstraction refers to, providing only essential information to the outside world and hiding their background details

Keywords/ Characteristics of OOP

Object

- This is the basic unit of object oriented programming. That is both data and function that operate on data are bundled as a unit called as object.

Class

- When you define a class, you define a blueprint for an object. This doesn't actually define any data, but it does define what the class name means, that is, what an object of the class will consist of and what operations can be performed on such an object.

Abstraction

- Data abstraction refers to, providing only essential information to the outside world and hiding their background details

Keywords/ Characteristics of OOP

- For example, a database system hides certain details of how data is stored and created and maintained. Similar way, C++ classes provides different methods to the outside world without giving internal detail about those methods and data.

Encapsulation

- Encapsulation is placing the data and the functions that work on that data in the same place. While working with procedural languages, it is not always clear which functions work on which variables but object-oriented programming provides you framework to place the data and the relevant functions together in the same object.

Difference between OOP and POP

- OOP treats data as a critical element in the program development and does not allow it to flow freely around the system.
- In OOP, the major emphasis is on data rather than procedure (function).
- It ties data more closely to the function that operate on it, and protects it from accidental modification from outside function.
- OOP allows decomposition of a problem into a number of entities called objects and then builds data and function around these objects.
- The data of an object can be accessed only by the function associated with that object. However, function of one object can access the function of other objects.
- C++, Java, Dot Net, Python etc are the example of Object oriented programming (OOP) language.

Difference between OOP and POP

- In the procedure oriented approach, large programs are divided into smaller programs known as functions.
- In POP, a program is written as a sequence of procedures or function.
- In POP, each procedure (function) contains a series of instructions for performing a specific task.
- While we concentrate onto the development of functions, we give very little attention to the data that are being used by various functions.
- In a multi-function program, many important data items are placed as global so that they may be accessed by all the functions. Each function may have its own local data.
- Examples : COBOL, FORTRAN, PASCAL, C programming language etc.

Difference between OOP and POP

Object Oriented Programming	Procedure Oriented Programming
<u>Object oriented.</u>	<u>Structure oriented.</u>
Program is divided into objects.	Program is divided into functions.
Bottom-up approach.	Top-down approach.
Inheritance property is used.	Inheritance is not allowed.
It uses access specifier.	It doesn't use access specifier.
Encapsulation is used to hide the data.	No data hiding.
Concept of virtual function.	No virtual function.
Object functions are linked through message passing.	Parts of program are linked through parameter passing.
Adding new data and functions is easy	Expanding new data and functions is not easy.
The existing code can be reused.	No code reusability.
use for solving big problems.	Not suitable for solving big problems.
<u>C++, Java.</u>	<u>C, Pascal.</u>

Use Cases and Applications

- OOP use cases are common in large, complex, and scalable applications where data security, modularity, and reusability matter. It is widely used in software like banking systems, CRMs, e-commerce platforms, mobile apps, and gaming engines.
- POP on the other hand works best for small scripts, mathematical computations, file handling tasks, and system-level programming.
- In OOP real-world examples, consider an e-commerce application: products, users, and orders are represented as objects with attributes and methods, making the system modular, reusable, and easier to maintain.
- In contrast, a POP real-world example could be a simple payroll system, where functions handle calculations sequentially without encapsulating data.

Structure of a C++ Program:

Documentation
Link Section
Definition Section
Global Declaration Section
Function definition Section
Main Function

Skeleton of C Program

Documentation Section:

- This section comes first and is used to document the logic of the program that the programmer going to code.
- It can be also used to write for purpose of the program.

Structure of a C++ Program:

Linking Section:

- The linking section contains two parts:

Header Files

- Generally, a program includes various programming elements like built-in functions, classes, keywords, constants, operators, etc. that are already defined in the standard C++ library.
- In order to use such pre-defined elements in a program, an appropriate header must be included in the program. **Ex: #include<iostream>**

Namespaces

- A namespace permits grouping of various entities like classes, objects, functions, and various C++ tokens, etc. under a single name. Any user can create separate namespaces of its own and can use them in any other program. **Ex: using namespace std;**

Structure of a C++ Program:

Definition Section:

- It is used to declare some constants and assign them some value.
- In this section, anyone can define your own datatype using primitive data types. **Ex**

Global Declaration Section:

- Here, the variables and the class definitions which are going to be used in the program are declared to make them global.
- The scope of the variable declared in this section lasts until the entire program terminates.
- These variables are accessible within the user-defined functions also.

Function Declaration Section:

- It contains all the functions which our main functions need.

Structure of a C++ Program:

Main Function:

- The main function tells the compiler where to start the execution of the program. The execution of the program starts with the main function.
- All the statements that are to be executed are written in the main function.
- The compiler executes all the instructions which are written in the curly braces {} which encloses the body of the main function.
- Once all instructions from the main function are executed, control comes out of the main function and the program terminates and no further execution occur.

Structure of a C++ Program:

- Usually, this section contains the User-defined functions.
- This part of the program can be written after the main function but for this, write the function prototype in this section. **Ex**

```
// Function to implement the  
// factorial of number num  
INTEGER factorial(INTEGER num)  
{  
    // Iterate over the loop from  
    // num to one  
    for (INTEGER i = 1; i <= num; i++) {  
        fact *= i;  
    }  
  
    // Return the factorial calculated  
    return fact;  
}
```

Declaration of Variables

- A variable provides us with named storage that our programs can manipulate. Each variable in C++ has a specific type, which determines the size and layout of the variable's memory; the range of values that can be stored within that memory; and the set of operations that can be applied to the variable.

- **C++ Variable Naming**

- The name of a variable can be composed of letters, digits, and the underscore character. It must begin with either a letter or an underscore. Upper and lowercase letters are distinct because C++ is case-sensitive.

Example of Valid Variable Names:

- `int age;`
- `int _age;`
- `int student_age;`
- `int studentAge;`

Declaration of Variables

Variable Definition in C++

- A variable definition tells the compiler where and how much storage to create for the variable. A variable definition specifies a data type, and contains a list of one or more variables of that type as follows –

type variable_list;

- type must be a valid C++ data type including char, w_char, int, float, double, bool or any user-defined object, etc., and variable_list may consist of one or more identifier names separated by commas

Variable Initialization in C++

- Variables can be initialized (assigned an initial value) in their declaration. The initializer consists of an equal sign followed by a constant expression as follows –

type variable_name = value;

Declaration of Variables

- A variable declaration provides assurance to the compiler that there is one variable existing with the given type and name so that compiler proceed for further compilation without needing complete detail about the variable. A variable declaration has its meaning at the time of compilation only, compiler needs actual variable definition at the time of linking of the program.
- A variable declaration is useful when you are using multiple files and you define your variable in one of the files which will be available at the time of linking of the program. You will use extern keyword to declare a variable at any place. Though you can declare a variable multiple times in your C++ program, but it can be defined only once in a file, a function or a block of code.

Declaration of Variables

Lvalues and Rvalues

- Variables are lvalues and so may appear on the left-hand side of an assignment. Numeric literals are rvalues and so may not be assigned and can not appear on the left-hand side.

Following is a valid statement –

```
int g = 20;
```

- But the following is not a valid statement and would generate compile-time error –

```
10 = 20;
```


Scope of Variables

- A scope is a region of the program and broadly speaking there are three places, where variables can be declared –
 - Inside a function or a block which is called local variables,
 - In the definition of function parameters which is called formal parameters.
 - Outside of all functions which is called global variables.

- C++ variables scopes are categorized mainly two parts –
 - Local Variables
 - Global Variables

Scope of Variables

```
#include <iostream>
using namespace std;
```

```
int g = 20;
```

Global Variables

```
int main () {
```

```
    int g = 10;
```

Local Variables

```
    cout << g;
```

```
    return 0;
```

```
}
```

Scope of Variables

Local Variables

- Variables that are declared inside a function or block are local variables. They can be used only by statements that are inside that function or block of code. Local variables are not known to functions outside their own.

Global Variables

- Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the life-time of your program.
- A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration.

Basic Data Types in C++

- While doing programming in any programming language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.
- You may like to store information of various data types like character, wide character, integer, floating point, double floating point, Boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.
- **Primitive Built-in Types**
- C++ offers the programmer a rich assortment of built-in as well as user defined data types

Basic Data Types in C++

Type	Keyword
<u>Boolean</u>	bool
<u>Character</u>	char
<u>Integer</u>	int
Floating point	float
Double floating point	double
Valueless	void
Wide character	wchar_t

Basic Data Types in C++

- Several of the basic types can be modified using one or more of these type modifiers –
 - signed
 - unsigned
 - short
 - Long

- The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Basic Data Types in C++

Type	Typical Bit Width	Typical Range
char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295
signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	2bytes	0 to 65,535
signed short int	2bytes	-32768 to 32767
long int	8bytes	-9223372036854775808 to 9223372036854775807
signed long int	8bytes	same as long int

Derived Data Types

- Data types which are obtained from pre-defined data types in C++ are known as

Derived Data Types.

- **Function:** A function is the simplest form of user-defined data type. It includes a return type, a function name and input parameters.

Array

- An array is a series of elements of same data type. Elements of an array are stored in contiguous memory locations in the storage.

Pointer

- A pointer is a reference to an element defined previously. The value of the pointer returns the address location of the element which is associated with it.

Reference

- A reference variable is used to create a copy of a variable with the same reference. Hence, changes made to the reference variable also reflect on the original variable.

User Defined Data Types

- Data types which are defined by the user intuitively without using any pre-defined data types are known as User-Defined Data Types. These data types can be further categorized into five types, namely –

Class

- A class is defined in Object Oriented Programming as a custom data type which is used to construct an object. It is the framework of an object, and it can include constructors, methods and OOP concepts like Polymorphism, Inheritance, etc.

- **Syntax**

```
class Class_name{  
<class body> class_name(parameters) {  
<constructor body> }  
return_type method_name(parameters){  
<method body> }  
}
```

User Defined Data Types

Structure (struct)

- In structure data type, the user can introduce multiple primitive data types inside the struct body.

- **Syntax**

```
struct struct_name{ data_type1 var_name1; data_type2 var_name2; }
```

Union

- Union is similar to a structure. In this, the memory location of all variables is same, and all variables share the same reference. Hence, a change in one value leads to all other values getting changed.

- **Syntax**

```
union union_name{ data_type var_name1; data_type var_name2; };
```

User Defined Data Types

Enumeration (Enum)

- Enumeration or simply enum is a user-defined data type that is used to give name to integer constants in a program. This increases the user-readability of a program.

- **Syntax**

```
enum enum_name{ var_name1 , var_name2, }
```

Reference Variables

- In C++, the reference variable is an alternate name of already existing variable. It cannot be changed to refer another variable and should be initialized at the time of declaration and cannot be NULL.
- The operator '&' is used to declare reference variable.
- Reference variables are commonly used for parameter passing in functions, returning values, and aliasing existing variables.

➤ Syntax

`datatype &refer_var = variable_name;`

Here

datatype - The datatype of variable like int, char, float etc.

variable_name - This is the name of variable given by user.

refer_var - The name of reference variable.

Reference Variables

- In the example, we can analyse that a variable of integer type is declared and initialized with a value. So, we can see that the variable b is declared which is referring variable a.

```
#include<iostream>  
using namespace std;  
int main() {  
    int a = 8;  
    int& b = a;  
    b = 20;  
    cout << "The variable a : " << a;  
    cout << "\nThe reference variable r : " << b;  
    return 0;  
}
```

Operators in C++

- Operators are symbols that perform operations on variables and values. For example, + is an operator used for addition, while - is an operator used for subtraction.
- Operators in C++ can be classified into 5 types:
 - Arithmetic Operators
 - Assignment Operators
 - Relational Operators
 - Logical Operators
 - Bitwise Operators

Operators in C++

Arithmetic Operators

- Arithmetic operators are used to perform arithmetic operations on variables and data.

a + b;

- Here, the + operator is used to add two variables a and b. Similarly there are various other arithmetic operators in C++

+ Addition

- Subtraction

* Multiplication

/ Division

% Modulo Operation

Operators in C++

Increment and Decrement Operators

- C++ also provides increment and decrement operators: ++ and -- respectively.
- ++ increases the value of the operand by 1
- -- decreases it by 1

For example,

```
int num = 5;
```

```
// increment operator
```

```
++num; // 6
```


Operators in C++

C++ Assignment Operators

➤ In C++, assignment operators are used to assign values to variables. For example,

```
// assign 5 to a  
a = 5;
```

Operator	Example	Equivalent to
=	<code>a = b;</code>	<code>a = b;</code>
+=	<code>a += b;</code>	<code>a = a + b;</code>
-=	<code>a -= b;</code>	<code>a = a - b;</code>
*=	<code>a *= b;</code>	<code>a = a * b;</code>
/=	<code>a /= b;</code>	<code>a = a / b;</code>
%=	<code>a %= b;</code>	<code>a = a % b;</code>

Operators in C++

C++ Relational Operators

- A relational operator is used to check the relationship between two operands. For example,

```
// checks if a is greater than b  
a > b;
```

- Here, > is a relational operator. It checks if a is greater than b or not.
- If the relation is true, it returns 1 whereas if the relation is false, it returns 0.

Operators in C++

C++ Relational Operators

Operator	Meaning	Example
<code>==</code>	Is Equal To	<code>3 == 5</code> gives us false
<code>!=</code>	Not Equal To	<code>3 != 5</code> gives us true
<code>></code>	Greater Than	<code>3 > 5</code> gives us false
<code><</code>	Less Than	<code>3 < 5</code> gives us true
<code>>=</code>	Greater Than or Equal To	<code>3 >= 5</code> give us false
<code><=</code>	Less Than or Equal To	<code>3 <= 5</code> gives us true

Operators in C++

C++ Logical Operators

- Logical operators are used to check whether an expression is true or false. If the expression is true, it returns 1 whereas if the expression is false, it returns 0.

Operator	Example	Meaning
&&	expression1 && expression2	Logical AND. True only if all the operands are true.
	expression1 expression2	Logical OR. True if at least one of the operands is true.
!	!expression	Logical NOT. True only if the operand is false.

Operators in C++

C++ Bitwise Operators

In C++, bitwise operators are used to perform operations on individual bits. They can only be used alongside char and int data types.

Operator	Description
&	Binary AND
	Binary OR
^	Binary XOR
~	Binary One's Complement
<<	Binary Shift Left
>>	Binary Shift Right

Operators in C++

Other C++ Operators

Here's a list of some other common operators available in C++. We will learn about them in later tutorials.

Operator	Description	Example
<code>sizeof</code>	returns the size of data type	<code>sizeof(int); // 4</code>
<code>?:</code>	returns value based on the condition	<code>string result = (5 > 0) ? "even" : "odd"; // "even"</code>
<code>&</code>	represents memory address of the operand	<code>&num; // address of num</code>
<code>.</code>	accesses members of struct variables or class objects	<code>s1.marks = 92;</code>
<code>-></code>	used with pointers to access the class or struct variables	<code>ptr->marks = 92;</code>
<code><<</code>	prints the output value	<code>cout << 5;</code>
<code>>></code>	gets the input value	<code>cin >> num;</code>

Access Specifiers in C++

- One of the main features of object-oriented programming languages such as C++ is data hiding.
- Data hiding refers to restricting access to data members of a class. This is to prevent other functions and classes from tampering with the class data.
- However, it is also important to make some member functions and member data accessible so that the hidden data can be manipulated indirectly.
- The access modifiers of C++ allows us to determine which class members are accessible to other classes and functions, and which are not.
- For example,

Access Specifiers in C++

```
class Patient {  
    private:  
        int patientNumber;  
        string diagnosis;  
    public:  
        void billing() {  
            // code  
        }  
        void makeAppointment() {  
            // code  
        }  
};
```

Here, the variables `patientNumber` and `diagnosis` of the `Patient` class are hidden using the `private` keyword, while the member functions are made accessible using the `public` keyword.

Access Specifiers in C++

Types of C++ Access Modifiers

In C++, there are 3 access modifiers:

- public
- private
- Protected

Public Access Modifier

- The public keyword is used to create public members (data and functions).
- The public members are accessible from any part of the program.

Private Access Modifier

- The private keyword is used to create private members (data and functions).
- The private members can only be accessed from within the class.
- However, friend classes and friend functions can access private members.

Access Specifiers in C++

Protected Access Modifier

- Before we learn about the protected access specifier, make sure you know about inheritance in C++.
- The protected keyword is used to create protected members (data and function).
- The protected members can be accessed within the class and from the derived class.

Parul[®]
University

NAAC
GRADE **A++**



<https://paruluniversity.ac.in/>

